ECEN 248 - Lab Report

Lab #5

Introduction to Logic Simulation and Verilog

Section: 524

Ramya Kannan

UIN: 635002011

10/23/2025

TA: Reed Clardy

Objectives:

The objective of this lab is to introduce topics regarding logical simulation and Verilog Hardware Description Language (HDL) as an alternative to physical simulation using the breadboard for the digital circuit design and testing. This lab demonstrates how to use Xilinx Vivado Design to simulate digital circuits without the physical construction. We will implement previously designed circuits (that we did physically) - 2:1 Mux, 4-bit Mux, full adder, addition/subtraction unit, and a 4-bit ALU using Verilog. The goal of this lab is to gain skills in writing Verilog code, and utilizing the testbenches for circuit verification, and understanding the advantage of simulation-based design.

Design:

The design and testing of this lab was demonstrated through digital simulation using Verilog HDL in the Xilinx Vivado Design Suite. The design process began with a 1-bit, 2:1 Mux module named two_one_mux with output Y and inputs A, B, and S declared as wires. Internal nets notS, andA, and andB held intermediate signals. The mux used gate-level primitives: not0 inverted S to create notS, and0 and and1 gated the inputs with the select signals, and or0 combined the results to produce Y. This was verified with two_one_mux_tb.v.

Next, the 4-bit, 2:1 mux module was created using structural Verilog. It took two 4-bit inputs A[3:0] and B[3:0] plus select signal S to produce output Y [3:0]. Four instances of two_one_mux (MUX[0:3]) were instantiated, each handling one bit position. This was tested with four_bit_mux_tb.v.

The full_adder module uses dataflow Verilog with assign statements. It had inputs A, B, and Cin, and outputs S and Cout. Internal wires andBCin, and ACin, and andAB held intermediate AND values. Sum was calculated as S = A ^ B ^ Cin, while the carry used three AND terms combined with OR (Cout = andAB | andBCin | andACin). This was verified with full_adder_tb.v.

Next, the add_sub module was created and used a 4-bit ripple-carry adder with outputs Sum[3:0] and Overflow, and inputs opA[3:0], opB[3:0], and opSel. Each bit of opB was XORed with opSel to create notB[3:0] - unchanged for addition (opSel = 0) or inverted for subtraction (opSel = 1). Four full adder instances (adder0-adder3) were connected with carries c0-c3 rippling through. The opSel also fed into adder0 as carry-in for the +1 in two's complement. Overflow was detected by XORing c2 and c3. This was tested with add_sub_tb.v.

Finally we implemented the four_bit_alu. This module was named four_bit_alu, and it combined everything. It had outputs Result[3:0] and Overflow, and inputs opA[3:0], opB[3:0], and ctrl[1:0]. Internal wires Sum[3:0], andAB[3:0], and OverflowInit connected the components. The add_sub instance (addsub) used ctrl[1] to control operation type. Four assign statements computed the bitwise AND (andAB[i] = opA[i] & opB[i]). A four_bit_mux instance selected between Sum and andAB based on ctrl[0], where ctrl[0] = 0 gave the AND result and ctrl[0] = 1 gave the arithmetic result. The final Overflow output was OverflowInit ANDed with ctrl[0] so it only flagged during arithmetic operations. This was verified with four_bit_alu_tb.v.

Results:

The modules we created and tested were successfully verified using Vivado through the waveform simulations using the testbenches that were provided.

First we tested the 1-bit 2:1 Mux, the testbench tested all eight possible input combinations for the inputs A, B, and S. The simulation created a waveform which showed that when the select signal S = 0, the output Y follows input A, and when S = 1, the output Y follows input B. THe testbench cycled through the tests over 80ns, and all the eight test cases passed.

Secondly, we tested the 4-bit 2:1 Mux, which implemented four instances of the 1-bit Mux. The testbench ran for 160ns, verifying that Y[3:0] correctly selected A[3:0] when S = 0, and B[3:0] when S = 1. All the test cases passed!

Thirdly, we simulated the full adder module using dataflow abstraction. The testbench ran for 80ns, meaning that all the 8 combinations of the three 1-bit inputs A, B, and Cin were tested as well as S (Sum) and Cout (carryout) were verified. An example demonstrating that the full adder was successfully tested is: when A = 1, B = 0, and Cin = 1, the sum S correctly produced 0 and Cout produced 1, representing the binary addition 1+ 0+ 1 = 10 in binary. All the test cases passed, verifying the full adder module.

Next, the additon/subtraction unit was tested. The testbench ran for 320 ns and there were 32 test cases that passed. For the 4-bit operands opA[3:0] and opB[3:0], various combinations were tested while also enabling and disabling the opSel. When opSel =0, addition was implemented by passing opB unchanged through the XOR gates and setting the carry-in to 0. The addition/subtraction module passed all the test cases and the overflow detection worked successfully.

Lastly, the 4-bit ALU module was tested. The test bench ran for 640 ns, 64 test cases passed. This module incorporated all the previous tested modules. Various combinations of opA[3:0], opB[3:0], and ctrl[1:0] were tested to verify all the operation modes. The four_bit_mux successfully selected between AND results (ctrl[0] = 0) and arithmetic results (ctrl[0] = 1) from the add_sub module. For ctrl = 11, it implemented two's complement subtraction with proper overflow detection, exactly like for the 4th simulation. And for when ctrl = 01, addition was demonstrated. The ALU module was successfully tested and verified.

Conclusion:

In conclusion, the 4-bit ALU was successfully designed and verified through Verilog simulation in Xilinx Vivado.. The ALU integrated multiple components (modules) including a 2:1 mux, full adder, and addition/subtraction unit, demonstrating hierarchical digital system design. Throughout this lab, we gained experience with different Verilog levels: gate-level primitives for the 1-bit MUX, structural Verilog for the 4-bit Mux, and dataflow modeling for the full adder. We successfully tested and verified all the modules, and they all passed their respective testbenches. Waveforms were generated for each module to visualize and verify each operation. This lab highlights the advantages of implementing simulation based design over physical breadboarding, since it allows for easier debugging and efficiency.

Post-Lab Deliverables:

1. Include the source code with comments for all modules you simulated. You do not have to include test bench code. Code without comments will not be accepted!

Figure 1. Code for Four Bit ALU

Project Summary | four_bit_alu.v

C:/Users/ramka/Documents/lab5/four_bit_alu.v

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module four_bit_alu(output wire [3:0] Result, output wire Overflow, input wire [3:0] opA, opB, input wire
5
6      wire [3:0] Sum; //how we did for the 4-bit Sum wire
7      wire [3:0] andAB; //tbe and of A and B
8      wire OverflowInit; //Temp overflow wire oause the ALu will produce and overflow for some combos.
9      //this ^ is from the add_sub module
10
11     //we have to initialize addsub (like how we did in the add_sub.v)
12     add_sub addsub(Sum, OverflowInit, opA, opB, ctrl[1]);
13
14     //overflow logic for 4 bit alu
15
16     //its only valud when ctrl[0] is high
17     assign Overflow = OverflowInit & ctrl[0];
18
19     //vals of A and B for each bit
20     //this creates the AND result for when ctrl[0] = 1
21     assign andAB[0] = opA[0] & opB[0];
22     assign andAB[1] = opA[1] & opB[1];
23     assign andAB[2] = opA[2] & opB[2];
24     assign andAB[3] = opA[3] & opB[3];
25
26     //initialize 4 bit 2:1 mux
27     //when ctrol[0] = 0 it select SUM (add/subtract result)
28     //when ctrl[0] = 1 it selects andAB (bitwose AND result)
29     four_bit_mux four_bit_mux0(Result, andAB, Sum, ctrl[0]);
30 endmodule
31
```

Figure 2. Addition/Subtraction Code

Project Summary | add_sub.v

C:/Users/ramka/Documents/lab5/add_sub.v

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module add_sub(
5
6      output wire [3:0] Sum,
7      output wire Overflow,
8      input wire [3:0] opA, opB, //the 4 bit opperands for A and B
9      input wire opSel); //changes to either add or sub
10
11     //declares the internal nets as we did in the other files (similar)
12     wire [3:0] notB;
13     wire c0, c1, c2, c3;
14
15     //implementing the complement logic
16
17     assign notB[0] = opB[0] ^ opSel;
18     assign notB[1] = opB[1] ^ opSel;
19     assign notB[2] = opB[2] ^ opSel;
20     assign notB[3] = opB[3] ^ opSel;
21
22     //implementing the ripply carry adder with the full adders
23     //takes in each of varibles
24     full_adder adder0(Sum[0], c0, opA[0], notB[0], opSel);
25     full_adder adder1(Sum[1], c1, opA[1], notB[1], c0);
26     full_adder adder2(Sum[2], c2, opA[2], notB[2], c1);
27     full_adder adder3(Sum[3], c3, opA[3], notB[3], c2);
28
29     //now we need to detect for any overflow like we did in thelab
30     assign Overflow = c2 ^ c3;
31 endmodule
```

Figure 3. 4-bit Mux Code

Project Summary | four_bit_mux.v

C:/Users/ramka/Documents/lab5/four_bit_mux.v

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module four_bit_mux(Y, A, B, S);
5
6      input wire [3:0] A, B;
7      input wire S;
8      output wire [3:0] Y; //need to implement output wire
9
10     two_one_mux MUX0(Y[0], A[0], B[0], S);
11     two_one_mux MUX1(Y[1], A[1], B[1], S);
12     //adding two more module instantiations
13     two_one_mux MUX2(Y[2], A[2], B[2], S);
14     two_one_mux MUX3(Y[3], A[3], B[3], S);
15
16 endmodule
17
```

Figure 4. Full Adder Code

Project Summary | full_adder.v

C:/Users/ramka/Documents/lab5/full_adder.v

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module full_adder(S, Cout, A, B, Cin);
5      //DECLARING OUTPUT AND INPUT PORTS
6      //1 BIT WIRES
7      input wire A, B, Cin; //1 BIT WIRES
8
9     //need to delcare outputs
10     output wire S, Cout;
11
12     //DECLARING INTERNAL NETS
13     wire andBCin, andACin;
14     //one more forgot it
15     wire andAB;
16
17     //use dataflow to descirbe the gate
18     assign S = A ^ B ^ Cin;
19     assign andAB = A & B;
20     //for BC and AC
21     assign andBCin = B & Cin;
22     assign andACin = A & Cin;
23     assign Cout = andAB | andBCin | andACin;
24
25
26 endmodule
```

Figure 5. 2-bit Mux

Project Summary | two_one_mux.v

C:/Users/ramka/Documents/lab5/two_one_mux.v

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module two_one_mux(Y, A, B, S); //define the module name
5      //declare output and input ports
6      output wire Y; //declare output of type wire
7      input wire A, B, S; //declare inputs of type wire
8
9      //declares internal nets
10     wire notS;
11     wire andA;
12     wire andB;
13
14     //*instatiates gate leel modules
15     not not0(notS, S);
16     and and0(andA, notS, A);
17     and and1(andB, S, B);
18     or or0(Y, andA, andB);
19 endmodule //designate end of endmodule
20
21
```

2. Include screenshots of all waveforms captured during simulation in addition to the test bench console output for each test bench simulation. Please ensure these are legible in your report. Waveforms need to be properly fit.

Figure 6. 2.1 MUX test bench Waveform

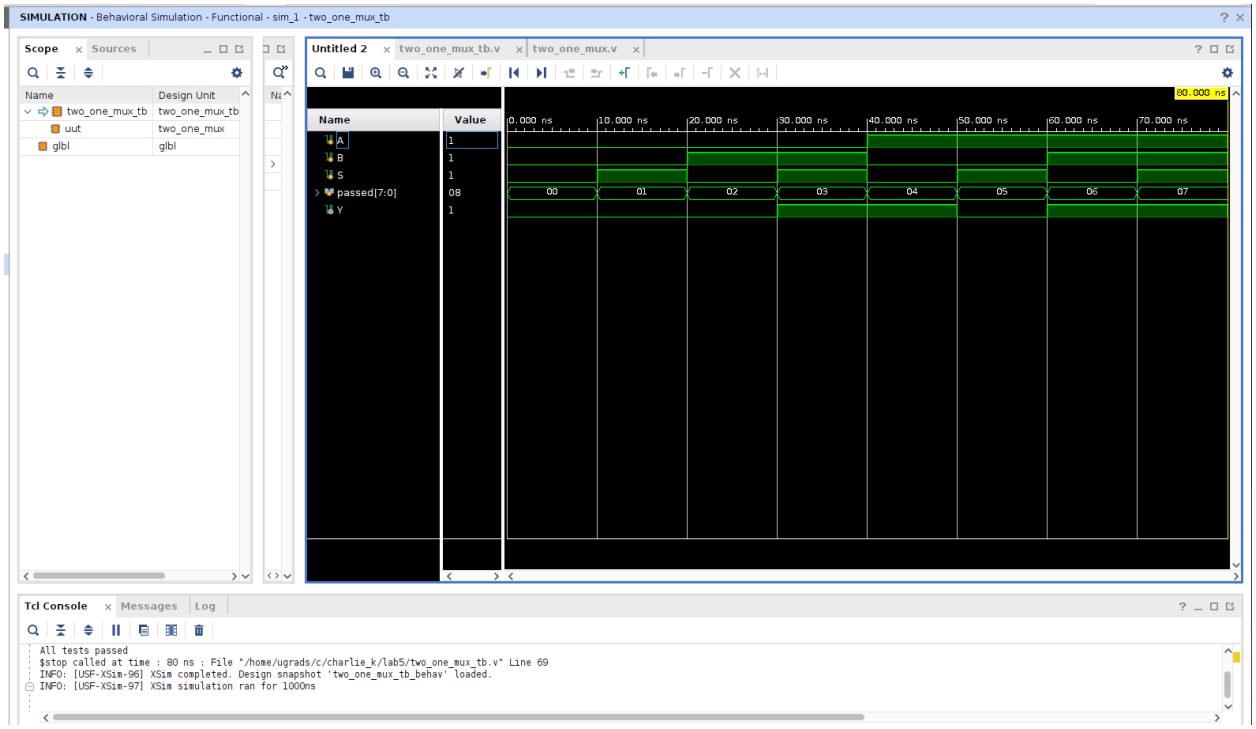


Figure 7. 4-bit Mux Waveform

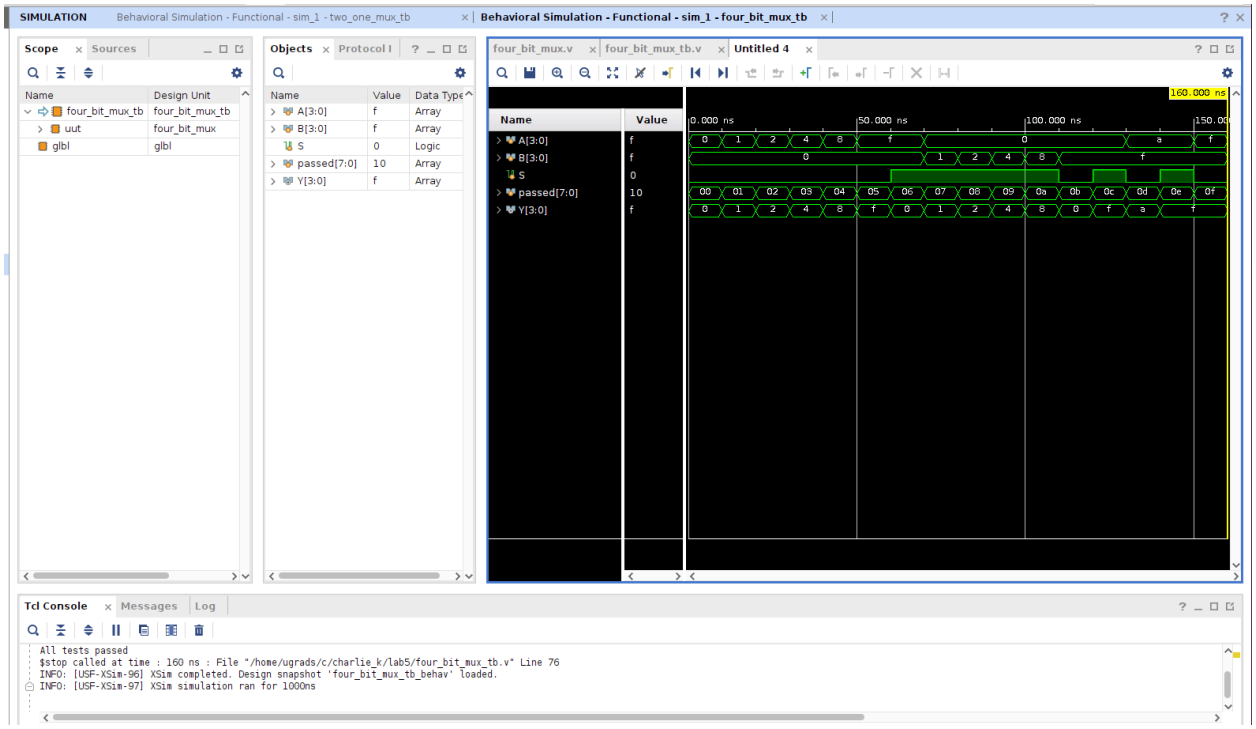


Figure 8. Full Adder Waveform

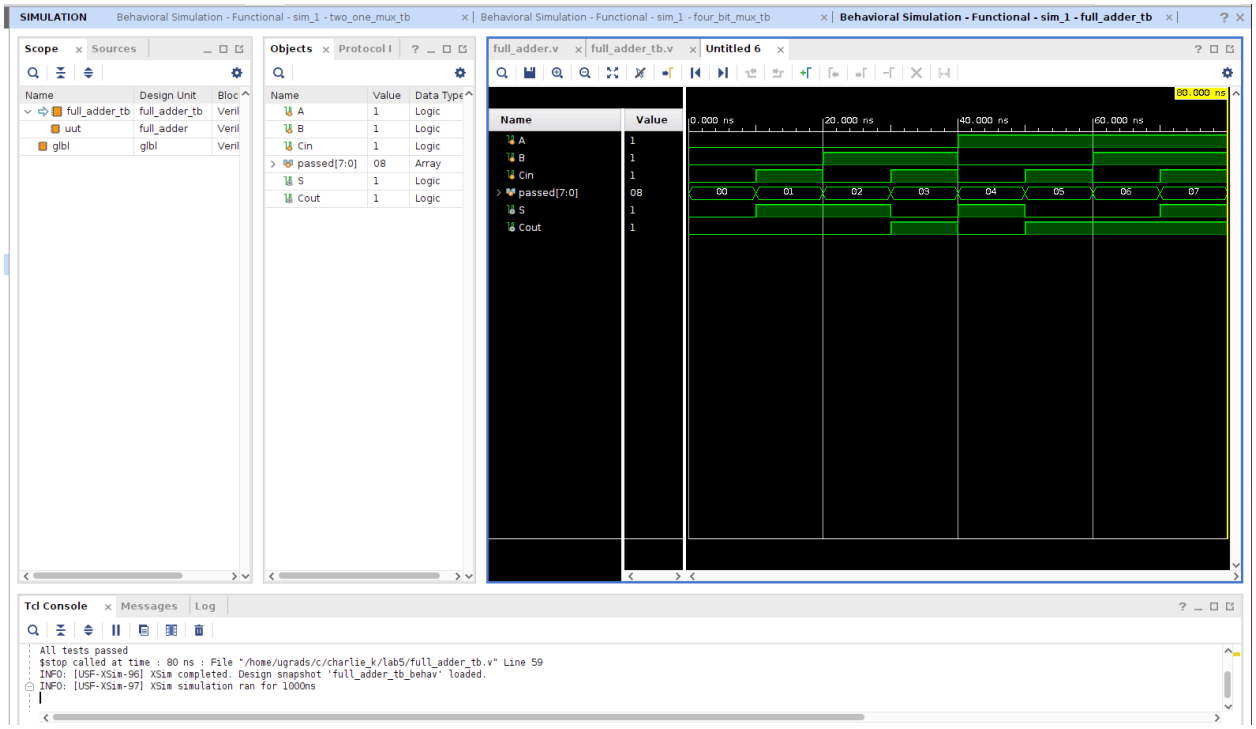


Figure 9. Addition/Subtraction Unit Waveform

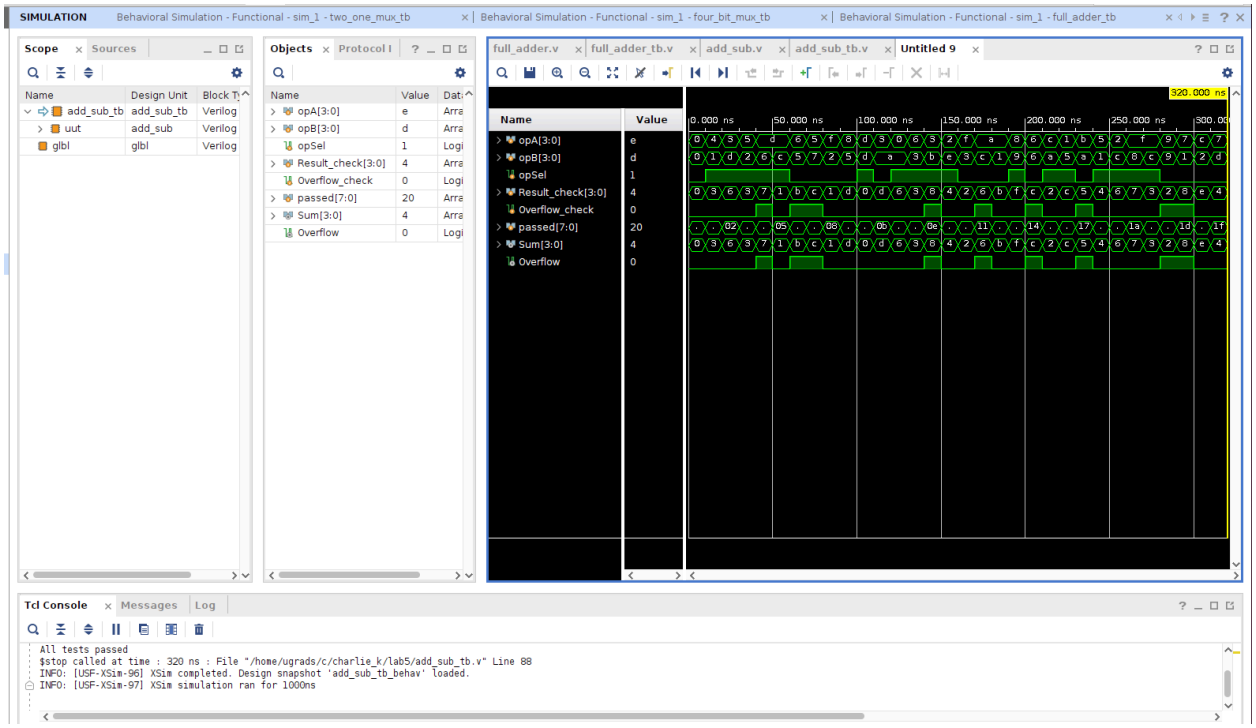
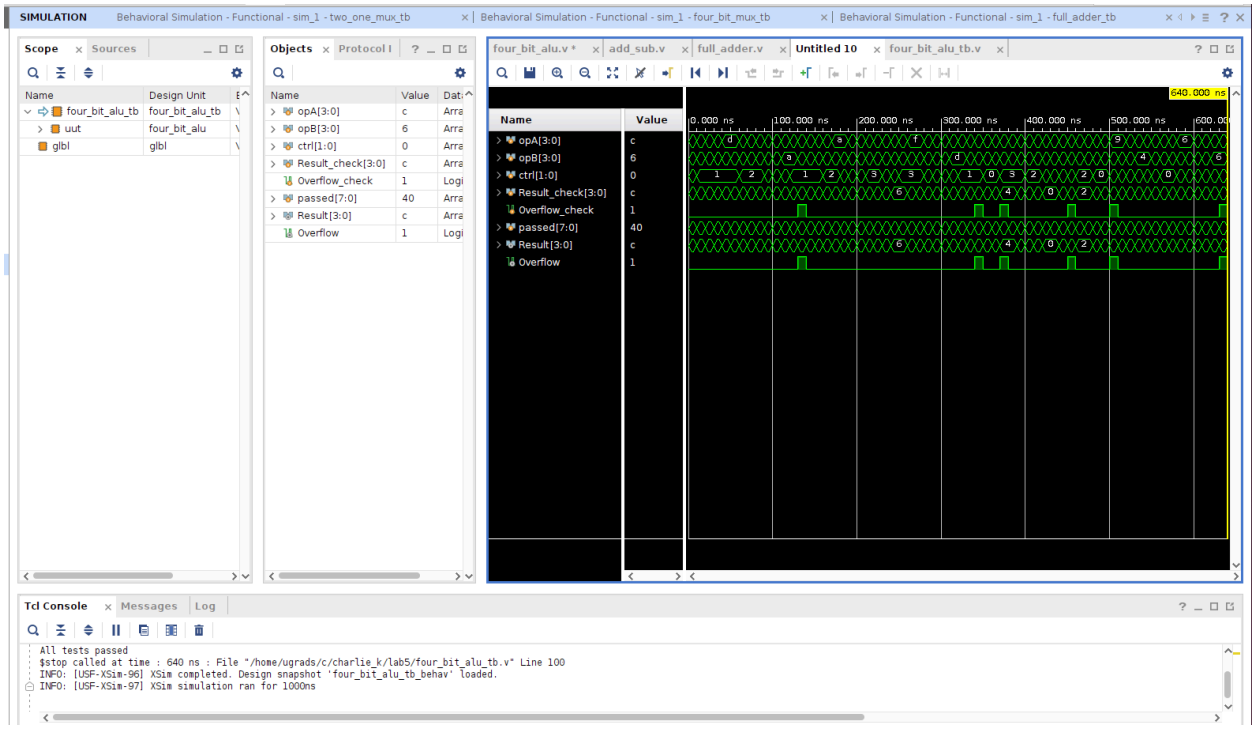


Figure 10. 4-bit ALU Waveform



3. Examine the 1-bit, 2:1 MUX test bench code. Attempt to understand what is going on in the code. The test bench is written using behavioral Verilog, which reads much more like a programming language. Explain briefly what it is the test bench is doing

The 2:1 MUX testbench for the 1-bit 2:1 mux creates an instance of the two_one_mux module and applies all the possible input combinations to verify the module's function. Using an initial block it assigns different values to input A, B, and S, then uses timing delays to allow the circuit to respond. For each test case, the testbench evaluates whether output Y correctly follows input A when S = 0 or input B when S = 1. The simulation runs through all eight combinations over 80ns, verifying the mux's behavior against the expected output. This testing approach verifies that the select signal controls which input is routed to the output.

4. Examine the 4-bit, 2:1 MUX test bench code. Are all of the possible input cases being tested? Why or why not?

No, the test bench code for the 4-bit 2:1 mux is not testing all the possible input cases. The 4-bit Mux has A[3:0], B[3:0], and S which is equivalent to 4 + 4 + 1 bits which is 2^9 = 512 possible combinations. To test all of these combinations is excessive, and since we already verified the 1-bit mux we only need to confirm that the four instances are wired together properly for the 4-bit 2:1 mux. The test bench tests multiple 4-bit values with both S = 0 and S = 1 to verify the structural connections. This is sufficient to verify the functionality of the 4-bit 2:1 mux without implementing excessive testing.

5. In this lab, we approached circuit design in a different way compared to previous labs. Compare and contrast breadboarding techniques with circuit simulation. Discuss the advantages and disadvantages of both. Which do you prefer? Similarly, provide some insight as to why HDLs might be preferred over schematics for circuit representation. Are there any disadvantages to describing a circuit using an HDL compared to a schematic? Again, which would you prefer?

Breadboarding: this technique provides hands-on learning with physical components such as DC powersupply, the breadboard itself, jumper wires, switches, LED's, etc. It enhances ones troubleshooting skills. You also get visuals demonstrating circuit behavior, for example using LED's as outputs. Although breadboarding is much easier to understand visually, it can be extremely time-consuming and prone to human error. Sometimes the wires can malfunction or break, or there are too many wires to keep track of.

Circuit Simulation: this technique is much more efficient and flexible. It's easier to debug, modify, and run tests automatically. We can visualize the signals through the output waveforms. We can also test WAY more cases faster through a circuit simulation, for example testing 64 ALu cases would be impractical on a breadboard. Through a simulation, we can see through the output and the waveform if tests have passed. One disadvantage through circuit simulation is that for people that like hands on learning, this wouldn't be ideal

HDLs vs. Schematics: HDLs support multiple abstraction levels and are easy to modify, debug, and reuse. It's more efficient for large designs, for example the hierarchical ALU we created in this lab, this would be super messy as a schematic. Also we can add comments to our code to keep track of what we are doing. Schematics provide us with a visualization of the circuit connections, which is helpful for simpler circuits. It's not as great for larger circuits because they can become cluttered and harder to debug.

Personally, I liked schematics because the visualization for the circuits was easy to understand, however I will say that when we got the ALU it was definitely harder to understand and I do prefer the HDL.

6. Two different levels of abstraction were introduced in this lab, namely structural and dataflow. Provide a comparison of these approaches. When might you use one over the other

Structural Verilog connects components by incorporating gates or modules. For example, the 1-bit 2:1 mux with gates NOT, AND, OR and the 4-bit mux by connecting four 1-bit mux instances [0:3]. It demonstrates how parts are wired together.

Dataflow Verilog uses assign statements with equations. We used dataflow for the full adder: S = A ^ B ^ Cin. It highlights what the circuit does mathematically rather than how its built.

You want to use structural when you are connecting existing modules into larger systems, like how we did for the 4-bit 2:1 Mux. And then you want to use dataflow for expressing logic through Boolean equations. This lab combined both of the approaches efficiently.

<u>Student Feedback:</u>

1. What did you like most about the lab assignment and why? What did you like least about it and why?

What I liked most about this lab assignment was that it was a lot cleaner and there weren't any broken wires. Coding made it easier to debug and verify the modules and its functionality. What I liked least about it was that visualizing it was different from breadboarding. I enjoyed how the breadboarding was LED outputs.

2. Were there any sections of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?

No, it was very clear.

3. What suggestions do you have to improve the overall lab assignment?

Maybe clarify in the lab what we need screenshots of.